

Date: 17-Nov-2025

Topics Covered

1. State ownership in React
 2. Lifting state up
 3. When not to lift state
 4. Global state management (Context, Redux, Zustand, Jotai)
-

1. State Belongs to a Component

- Each component owns its own state.
- Example:

```
function App() {  
  const [count, setCount] = useState(0);  
}
```

- `count` is private to `App`.
- Other components can only read it via props.

Key Principles: - State is local to the component instance. - Keep state close to where it is used. - Avoid unnecessary re-renders by not lifting state too early.

Discussion Notes: - We discussed how React keeps state separate from DOM. - Emphasized that props allow **unidirectional data flow**.

2. Lifting State Up

- Needed when multiple components need shared state.

Example:

```
function Calculator() {  
  const [temperature, setTemperature] = useState("");  
  
  return (  
    <div>  
      <TemperatureInput temperature={temperature} onChange={setTemperature} />  
    </div>  
  );  
}
```

```
        <BoilingVerdict temperature={temperature} />
      </div>
    );
  }
}
```

Key Points: - Parent owns state. - Child communicates changes via callback. - Avoid prop drilling if possible.

Discussion Notes: - Visualized data flow: Parent → Children via props, Children → Parent via callbacks. - Talked about how lifting state prevents multiple independent sources of truth.

3. When NOT to Lift State

- Keep state local if only one component needs it.

Example:

```
function ToggleButton() {
  const [on, setOn] = useState(false);
  return <button onClick={() => setOn(!on)}>{on ? "ON" : "OFF"}</button>;
}
```

Key Takeaways: - Over-lifting causes unnecessary re-renders. - Use React.memo or derived state patterns if lifting is unavoidable.

Discussion Notes: - We explored examples where lifting state would be unnecessary. - Emphasized keeping state close to the component that uses it.

4. Global State in Large Apps

- Prop drilling becomes a problem when state is needed deeply in nested components.
- Example hierarchy: App > Content > ProductList > ProductCard

Solutions:

4.1 Context API

```
const UserContext = createContext();

function App() {
  const [user, setUser] = useState({ name: "Faisal" });
```

```
    return (  
      <UserContext.Provider value={user}>  
        <Header />  
        <Content />  
      </UserContext.Provider>  
    );  
  }  
  
  function ProductCard() {  
    const user = useContext(UserContext);  
    return <p>Hello, {user.name}</p>;  
  }  
}
```

- Medium-sized apps: avoids prop drilling effectively.

4.2 Advanced Global State Libraries

- Redux, Zustand, Jotai.
- Allow any component to access and update state directly.
- Useful for complex, large-scale apps.

Discussion Notes: - We discussed trade-offs between Context API and state libraries. - Emphasized **performance optimizations** in large apps.

5. Key Takeaways

1. State belongs to the component that uses it.
 2. Lift state **only when multiple components need it**.
 3. Keep state local whenever possible.
 4. For large apps, use Context or global state libraries to manage shared state efficiently.
-

Code Snippets, Examples, and Visualizations - Included throughout the discussion in the code blocks above. - All examples were aligned with React best practices.

End of Session Notes